FULL-STACK CODING ASSESSMENT

Multi-Tenant Appointment & Booking SaaS

A production-oriented assessment specification for building a secure, scalable service-business booking platform

Assessment Attribute	Specification
Primary objective	Build a multi-tenant appointment booking and management SaaS
Target businesses	Barbers, salons, clinics, photographers, consultants, and similar service businesses
Frontend	React — deployed on Vercel
Backend	Node.js with Express or equivalent framework / Vercel serverless functions
Database & authentication	Supabase — PostgreSQL, Supabase Auth, Row-Level Security
Billing	Stripe or equivalent subscription flow; simulation acceptable
Core focus	Security, tenancy isolation, scheduling correctness, UX, API design, deployment
Assessment style	End-to-end implementation with documented architecture and trade-offs

Version 1.0 • Detailed Candidate Specification

Document Contents

1. 1. Assessment Overview
2. 2. Business Problem & Product Vision
3. 3. Goals, Non-Goals & Success Criteria
4. 4. Required Technology Stack
5. 5. System Architecture
6. 6. Multi-Tenant Architecture & Authentication
7. 7. Database Design & Schema
8. 8. Row-Level Security Requirements
9. 9. Public Booking Experience
10. 10. Availability & Scheduling Engine
11. 11. Booking Lifecycle & State Management
12. 12. Business Admin Dashboard
13. 13. Customer CRM
14. 14. Automated Reminders & Background Jobs
15. 15. Subscription Billing
16. 16. API Requirements
17. 17. Frontend Requirements
18. 18. Validation, Error Handling & Security
19. 19. Performance & Scalability
20. 20. Observability & Auditability
21. 21. Optional AI Enhancements
22. 22. Testing Requirements
23. 23. Deployment & Environment Configuration
24. 24. GitHub Repository & README Requirements
25. 25. Candidate Deliverables
26. 26. Assessment Rubric
27. 27. Suggested Implementation Plan
28. 28. Acceptance Criteria
29. 29. Bonus Challenges
30. 30. Final Submission Checklist

1. Assessment Overview

The candidate is expected to design and implement a complete multi-tenant appointment and booking platform for service-based businesses. The platform must support independent business accounts, public booking pages, service configuration, provider availability, appointment management, customer records, reminders, and recurring subscription access.

This assessment is intentionally broader than a CRUD exercise. The most important engineering challenges are tenant isolation, authorization, reliable availability calculation, prevention of double-booking, clean API boundaries, deployment readiness, and maintainable code structure.

Expected mindset: Treat the solution as a small production SaaS rather than a prototype. Explain important assumptions, security decisions, failure modes, and trade-offs.

2. Business Problem & Product Vision

Many service businesses manage appointments using phone calls, messaging applications, spreadsheets, or generic calendar tools. The proposed SaaS provides each business with its own branded public booking experience and private operations dashboard.

2.1 Core user types

User	Primary needs	Access
Business Owner / Admin	Configure business, services, staff/provider availability, bookings, clients, subscription	Authenticated tenant-scoped portal
Public Client	Browse services, choose date/time, submit contact information, receive confirmation	Public booking flow; no tenant-private data
System / Scheduler	Execute reminders and maintenance jobs	Server-side trusted execution context

2.2 Product vision

A business owner should be able to sign up, configure a business profile and services, publish a booking URL, and immediately accept appointments. A client should be able to complete a booking without creating an account unless the candidate intentionally adds that capability.

3. Goals, Non-Goals & Success Criteria

3.1 Goals

- Implement strict tenant isolation at the database authorization layer.
- Provide a simple public booking URL such as /booking/:business-slug.
- Calculate available slots from business/provider hours, service duration, blocked periods, and existing bookings.
- Prevent overlapping bookings and handle concurrent booking attempts safely.
- Provide business owners with a useful calendar and booking-management interface.
- Maintain a centralized customer CRM with history, notes, and lifetime value.
- Implement a realistic monthly subscription-access flow.
- Deploy the application using Vercel and Supabase.
- Provide a reproducible local setup through .env.example and README documentation.

3.2 Non-goals

- A full accounting system.
- A complete payroll or staff-management suite.
- A production-grade medical records system for clinics.
- Guaranteed real-world SMS delivery; simulation or provider abstraction is acceptable.
- A complete machine-learning platform for optional AI features.

3.3 Success criteria

The system is successful when a reviewer can create two businesses, configure each independently, publish both booking pages, create bookings, and verify that neither business can access the other's private data. The reviewer should also be able to change availability, see slots update correctly, manage appointments, and exercise subscription gating.

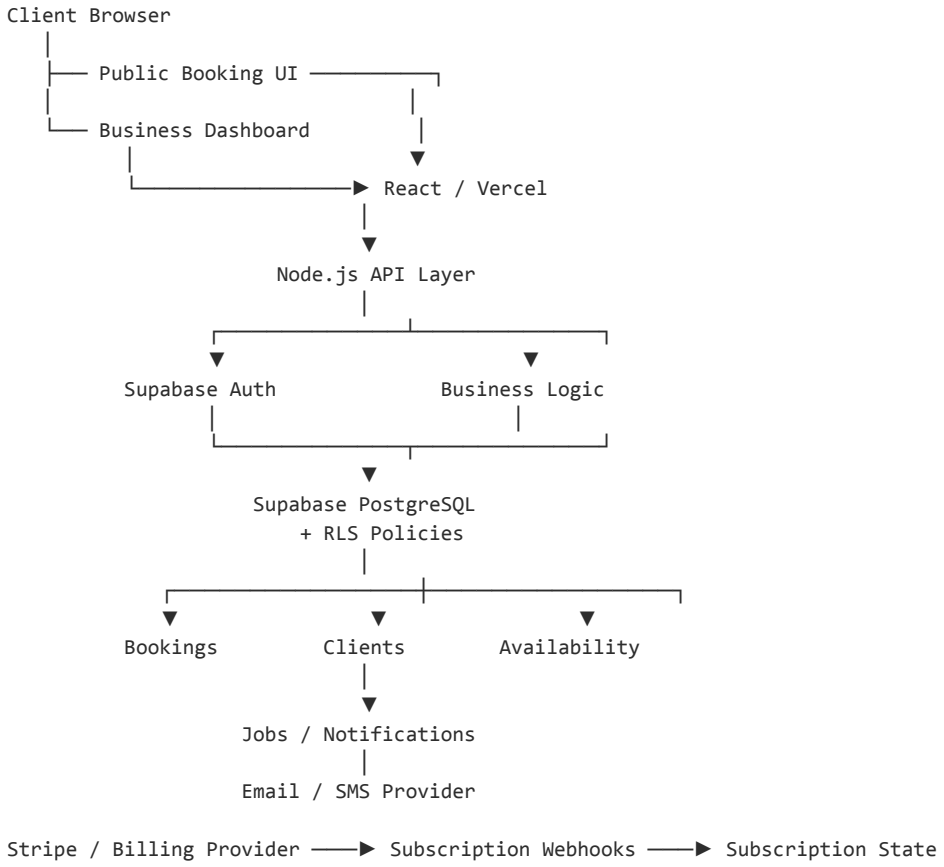
4. Required Technology Stack

Layer	Required / Preferred	Notes
Frontend	React	React Router or equivalent routing; componentized UI
Hosting	Vercel	Frontend deployment required; backend may use serverless/API routes
Backend	Node.js	Express or similar framework acceptable
Database	Supabase PostgreSQL	Relational schema strongly preferred
Authentication	Supabase Auth	Business users authenticate through Supabase
Authorization	PostgreSQL RLS	Must enforce tenant isolation at database layer
Billing	Stripe or equivalent	Monthly subscription; simulation accepted
Jobs	Vercel Cron / Edge Functions / equivalent	24-hour reminder processing
Email/SMS	Provider or mock adapter	Provider abstraction encouraged
Testing	Candidate choice	Unit + integration + critical E2E flows expected

Important: Do not rely solely on frontend checks to enforce authorization. UI hiding is not security.

5. System Architecture

A recommended high-level architecture is:



5.1 Architectural principles

- Tenant context must be explicit and derived from the authenticated identity or validated public business slug.
- Database RLS must provide a second line of defense even if an API endpoint is accidentally exposed.

- Booking creation must be transactional or otherwise protected against race conditions.
- External provider calls should be isolated behind service/adaptor modules.
- Business logic should not be duplicated between multiple frontend screens.
- Secrets must never be shipped to the browser.

6. Multi-Tenant Architecture & Authentication

6.1 Tenant model

Each business is a tenant. Every tenant-owned record must carry a business_id (or equivalent tenant key) either directly or through a parent relation. Tenant-owned data includes services, providers, schedules, blocked periods, clients, bookings, notifications, and subscription records.

6.2 Authentication

- Business owners sign up and log in using Supabase Auth.
- The authenticated user must be mapped to a business/tenant membership record.
- A user must not be able to select an arbitrary business_id from the browser and gain access to that tenant.
- Public clients do not need business-portal authentication for the basic booking flow.
- Admin-only actions require an authenticated business user and tenant authorization.

6.3 RBAC

Capability	Public Client	Business Owner/Admin
View public business profile	Yes	Yes
View public services	Yes	Yes
View available slots	Yes	Yes
Create public booking	Yes	Yes
View private bookings	No	Yes — own tenant
Edit services	No	Yes
Edit schedules	No	Yes
View CRM	No	Yes
Manage subscription	No	Yes
Access other tenant data	No	No

7. Database Design & Schema

The following schema is a recommended baseline. Candidates may rename or restructure tables if equivalent behavior and isolation are preserved.

Table	Key fields	Purpose
businesses	id, owner_user_id, name, slug, timezone, phone, email, status, created_at	Tenant/business profile
business_members	id, business_id, user_id, role, created_at	Tenant membership + RBAC
services	id, business_id, name, description, duration_minutes, price, active	Bookable services
providers	id, business_id, name, active	Staff/provider resources
provider_services	provider_id, service_id	Provider capability mapping
business_hours	id, business_id, day_of_week, start_time, end_time, enabled	Default operating hours
provider_hours	id, provider_id, day_of_week, start_time, end_time, enabled	Provider-specific hours
blocked_periods	id, business_id/provider_id, starts_at, ends_at, reason	Unavailable time
clients	id, business_id, name, email, phone, notes, created_at	Tenant CRM records

bookings	id, business_id, service_id, provider_id, client_id, starts_at, ends_at, status, price, source, created_at	Appointment records
booking_events	id, booking_id, event_type, actor_id, metadata, created_at	Audit/history trail
reminders	id, booking_id, channel, scheduled_for, sent_at, status	Notification jobs
subscriptions	id, business_id, provider_customer_id, provider_subscription_id, plan, status, current_period_end	Billing access state
notification_logs	id, business_id, booking_id, channel, template, status, provider_message_id, created_at	Delivery audit

7.1 Data integrity expectations

- Foreign keys should be used wherever practical.
- Use timestamptz for appointment timestamps and store the business timezone separately.
- Prices should use a safe representation such as integer minor units or a carefully defined numeric type.
- Booking status should be constrained to a known set of values.
- Slugs should be unique.
- Email/phone normalization should be considered for CRM deduplication.
- Indexes should be added for business_id, booking date ranges, client lookups, and public slug lookups.

8. Row-Level Security Requirements

RLS is a mandatory assessment requirement. Policies must make it impossible for an authenticated user from Business A to read or modify Business B's tenant-owned records through Supabase APIs.

8.1 Required policy strategy

- Determine the authenticated user's tenant membership using auth.uid().
- Allow SELECT/INSERT/UPDATE/DELETE only when the row's business_id belongs to that user.
- For child tables, enforce access through their business_id or a secure parent relationship.
- Public booking reads should expose only intentionally public business/service/scheduling information.
- Never expose private client notes, internal booking metadata, or subscription details through public policies.
- Test RLS using at least two distinct business users.

Security test: A reviewer should be able to attempt direct database/API queries using Business A credentials for Business B IDs and receive no unauthorized rows or mutations.

9. Client-Facing Public Booking System

9.1 Public route

```
/booking/:business-slug
```

The page should identify the business and present a focused booking journey.

9.2 Booking journey

31. Resolve business by slug.

32. Display business name, branding/profile information, operating context, and active services.
33. Client selects a service.
34. Client optionally selects a provider if multiple providers are exposed.
35. Client chooses a date.
36. System generates available time slots.
37. Client selects a slot.
38. Client enters name, email, phone, and optional notes.
39. System revalidates the slot on the server.
40. System creates the booking atomically if still available.
41. Show confirmation with appointment details.
42. Trigger confirmation notification through the notification adapter.

9.3 Public UX expectations

- Mobile-first responsive design.
- Clear service duration and price.
- Visible timezone/context where useful.
- Unavailable dates/slots should be clearly disabled rather than silently failing.
- Loading, empty, validation, and server-error states must be handled.
- Do not trust availability computed only on the client.

10. Availability & Scheduling Engine

Availability is one of the most important parts of the assessment. The implementation must account for service duration, business/provider hours, existing bookings, blocked periods, and timezone.

10.1 Recommended algorithm

43. Load the selected business and validate that it is active and bookable.
44. Load service duration.
45. Determine the provider/resource if provider selection is required.
46. Determine business/provider hours for the requested date.
47. Convert schedule boundaries into the business timezone and compare using canonical timestamps.
48. Load blocked periods overlapping the target date.
49. Load existing bookings that overlap the target date and exclude cancelled/expired statuses as appropriate.
50. Generate candidate start times at a defined slot interval (for example 15 or 30 minutes).
51. For each candidate, calculate candidate_end = candidate_start + service_duration.
52. Reject any candidate outside working hours.
53. Reject any candidate overlapping blocked periods.
54. Reject any candidate overlapping an active booking for the same provider/resource.
55. Return the remaining slots.

10.2 Overlap rule

```
Two appointments overlap when: existing_start < requested_end AND existing_end >
requested_start
```

10.3 Double-booking prevention

Availability display is not sufficient. The booking endpoint must re-check availability immediately before insertion. The database should provide the final consistency guarantee using a transaction,

exclusion constraint, advisory lock, or another concurrency-safe strategy appropriate to the chosen schema.

Edge cases to test: Two clients booking the same slot simultaneously; service duration crossing a slot boundary; daylight-saving changes; provider-specific hours; blocked periods; cancelled bookings; and bookings ending exactly when another begins.

11. Booking Lifecycle & State Management

Status	Meaning	Typical transitions
pending	Created but awaiting business action if approval is used	pending → confirmed/cancelled
confirmed	Appointment is accepted	confirmed → completed/cancelled/no_show
cancelled	Appointment cancelled	Terminal
completed	Service completed	Terminal
no_show	Client did not attend	Terminal

11.1 Required operations

- Create booking.
- View booking details.
- List bookings by day/week/month.
- Confirm booking where approval workflow exists.
- Cancel booking.
- Reschedule booking.
- Mark completed.
- Mark no-show.
- Record booking events.

11.2 Rescheduling

Rescheduling must run the same availability validation as a new booking. The old time must not remain blocked after a successful reschedule. The operation should be auditable.

12. Business Admin Dashboard

12.1 Calendar

- Daily, weekly, and monthly views.
- Appointments visually grouped by date/time and provider where applicable.
- Status indicators.
- Click/tap appointment to open detail.
- Actions for confirm, cancel, reschedule, complete, and no-show.
- Useful empty states.

12.2 Dashboard summary

Metric	Definition
Today's appointments	Count of appointments scheduled for current business day
Upcoming bookings	Near-term active appointments
Revenue / booking value	Sum of applicable booking prices; define whether cancelled records are excluded
Clients	Distinct CRM clients

No-show rate	No-show bookings divided by completed/expected appointments, with documented formula

12.3 Settings

- Business profile and slug.
- Services and prices.
- Provider management.
- Business/provider hours.
- Blocked periods.
- Notification preferences.
- Subscription status.

13. Customer CRM

The CRM should turn booking records into useful customer context without exposing data across tenants.

CRM field	Requirement
Name	Required for normal booking
Email	Validated where supplied
Phone	Validated/normalized where supplied
Notes	Private business notes; never public
Booking history	List of historical appointments
Lifetime value	Derived from qualifying booking value
Last appointment	Most recent completed/eligible booking
Next appointment	Nearest upcoming active booking

13.1 CRM behavior

- Search by name, email, or phone.
- Open a customer profile.
- View booking history.
- Add/edit internal notes.
- Show aggregate metrics.
- Avoid creating duplicate client records when the same customer books repeatedly.

14. Automated Reminders & Background Jobs

Implement a job that identifies appointments due for a 24-hour reminder. A real CRON or Edge Function is preferred; a clearly documented simulation is acceptable for an assessment.

14.1 Job behavior

56. Run on a recurring schedule.
57. Find eligible upcoming bookings around the 24-hour threshold.
58. Exclude cancelled bookings.
59. Check whether a reminder has already been sent.
60. Create/send the notification.
61. Record success/failure in reminder or notification logs.
62. Ensure the job is idempotent.

14.2 Failure handling

- Provider failure must not corrupt the booking.
- Retries should be bounded and observable.
- A failed notification should be recorded.
- Duplicate sends should be prevented by a database uniqueness rule or idempotency mechanism.

15. Monthly Subscription Billing

Business owners should have a SaaS subscription state that controls access to premium dashboard functionality. Stripe is recommended, but an equivalent provider or simulated billing implementation is acceptable if the assessment environment cannot support live payments.

15.1 Subscription states

State	Expected behavior
active	Full subscribed dashboard access
trialing	Full or configured trial access
past_due	Grace-period behavior documented by candidate
cancelled	Access removed at defined point
incomplete	Subscription setup not finished

15.2 Billing integration expectations

- Create or identify a provider customer for each business.
- Start a monthly subscription.
- Store provider identifiers server-side.
- Handle webhook events.
- Verify webhook signatures.
- Synchronize subscription status into the database.
- Do not trust a client-provided subscription status.
- Gate premium features using server-side state.

16. API Requirements

The exact API shape is flexible, but the following capabilities should exist.

Method	Example route	Purpose
GET	/api/public/businesses/:slug	Public business profile
GET	/api/public/businesses/:slug/services	Active public services
GET	/api/public/businesses/:slug/availability	Available slots
POST	/api/public/businesses/:slug/bookings	Create public booking
GET	/api/bookings	Tenant booking list
GET	/api/bookings/:id	Booking detail
PATCH	/api/bookings/:id	Update booking/status
POST	/api/bookings/:id/reschedule	Reschedule booking
GET	/api/clients	CRM list/search
GET	/api/clients/:id	CRM profile
PATCH	/api/clients/:id	Update client
GET	/api/services	Tenant service management
POST	/api/services	Create service
PATCH	/api/services/:id	Update service
GET	/api/schedule	Hours and availability settings
POST	/api/billing/checkout	Start subscription flow
POST	/api/billing/webhook	Provider webhook

16.1 API design expectations

- Consistent JSON response shape.
- Appropriate HTTP status codes.
- Server-side validation.
- Authentication middleware for private routes.
- Tenant authorization for every tenant-scoped operation.
- No sensitive secrets in responses.
- Useful error messages without leaking internal stack traces.

17. Frontend Requirements

17.1 Recommended routes

Route	Audience	Purpose
/	All	Product/landing page or redirect
/login	Business	Login
/signup	Business	Business onboarding
/dashboard	Business	Overview
/dashboard/calendar	Business	Calendar
/dashboard/bookings	Business	Booking list
/dashboard/clients	Business	CRM
/dashboard/services	Business	Service configuration
/dashboard/settings	Business	Business/schedule/settings
/dashboard/billing	Business	Subscription
/booking/:business-slug	Public	Booking page

17.2 UX quality bar

- Responsive across desktop and mobile.
- Consistent typography, spacing, buttons, forms, and status indicators.
- Keyboard-accessible interactive controls where practical.
- Clear confirmation after mutations.
- Optimistic UI only when safe; otherwise use server-confirmed state.
- Prevent accidental duplicate submits.
- Show meaningful skeleton/loading states.

18. Validation, Error Handling & Security

18.1 Validation

- Validate request bodies and query parameters server-side.
- Validate UUID/ID formats.
- Validate service duration and price boundaries.
- Validate date/time ranges.
- Validate business slug uniqueness and allowed characters.
- Normalize email and phone where appropriate.

18.2 Security checklist

- Supabase RLS enabled for tenant tables.
- Service-role keys never exposed to the client.
- Secrets stored as Vercel environment variables.
- Webhook signature verification enabled.

- Authorization enforced server-side.
- Public endpoints expose minimum necessary data.
- Rate limiting or basic abuse controls considered for public booking endpoints.
- Input sanitization and safe rendering of notes/user-generated text.
- CORS configured intentionally.
- Audit logs for sensitive booking state changes.

Threat model: Consider malicious tenant users, manipulated business IDs, replayed webhooks, duplicate booking submissions, leaked public URLs, and abusive public booking requests.

19. Performance & Scalability

- Avoid N+1 queries in calendar and CRM screens.
- Index common filters such as business_id + starts_at.
- Paginate large booking/client lists.
- Cache or memoize public business/service configuration where safe.
- Keep availability queries bounded to the requested date/range.
- Avoid loading all historical bookings merely to render today's calendar.
- Use database aggregation for lifetime value and counts when appropriate.

19.1 Expected scale discussion

The candidate should document how the design would change from hundreds to thousands of tenants. The answer should address database indexing, connection limits, job execution, notification throughput, and possible caching.

20. Observability & Auditability

- Structured server logs for important operations.
- Booking creation/cancellation/reschedule events recorded.
- Subscription webhook events traceable.
- Notification attempts recorded.
- Errors should include a correlation/request identifier where practical.
- Do not log authentication tokens, payment secrets, or unnecessary personal information.

21. Optional AI Enhancements

These features are optional and should never compromise core security or booking correctness.

21.1 AI Smart Scheduling Assistant

Provide a conversational interface that converts natural language into scheduling constraints. Example intent: "Find me a haircut slot next Tuesday afternoon." The assistant should identify service, date, time window, and optionally provider, then call the same authoritative availability API used by the normal UI.

- AI must not invent availability.
- Final booking must still use the server-side booking endpoint.
- Display the interpreted constraints before confirmation when ambiguity exists.

21.2 AI No-Show Predictor

Generate a heuristic risk score using historical booking behavior such as prior no-shows, cancellations, lead time, and booking frequency. Clearly label this as a prediction rather than fact.

21.3 Automated AI Follow-ups

Generate personalized post-appointment messages using service type, customer history, and business tone. The candidate should add safeguards to prevent inappropriate or excessive messaging.

22. Testing Requirements

22.1 Minimum tests

- Authentication success/failure.
- Tenant isolation between two businesses.
- Public business lookup by slug.
- Availability generation.
- Booking creation.
- Double-booking race/concurrency scenario.
- Cancellation and rescheduling.
- CRM history and lifetime value.
- Reminder idempotency.
- Subscription access gating.
- Webhook signature rejection.

22.2 Recommended test matrix

Scenario	Expected result
Business A requests Business B private clients	Denied / empty according to policy
Two users book identical provider/time	At most one succeeds
Cancelled booking occupies slot	Slot becomes available
Service duration = 90 min	90-minute overlap is checked
Blocked period overlaps candidate	Slot rejected
Booking exactly ends at next booking	No overlap if boundary rule allows it
Reminder job runs twice	Only one reminder sent
Invalid webhook signature	Rejected
Expired subscription accesses premium API	Denied according to policy

23. Deployment & Environment Configuration

23.1 Vercel deployment

- Frontend deployed on Vercel.
- Backend/API deployed through Vercel serverless functions or an equivalent Node deployment compatible with the required stack.
- Production build must succeed from a clean checkout.
- Environment variables configured through Vercel rather than committed to Git.

23.2 Required .env.example

```
# Supabase
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# Application
APP_URL=
API_URL=
```

```
# Billing
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
STRIPE_PRICE_ID=

# Notifications
EMAIL_PROVIDER_API_KEY=
SMS_PROVIDER_API_KEY=

# Jobs
CRON_SECRET=
```

Important: Only variables genuinely required by the implementation should be retained. The example above is a reference, not a demand to use every provider.

24. GitHub Repository & README Requirements

24.1 Repository quality

- Clear folder structure.
- Meaningful commit history.
- No secrets or .env files committed.
- No unnecessary generated artifacts.
- Consistent formatting/linting.
- Useful comments around non-obvious business logic.

24.2 README must include

63. Project overview.
64. Architecture diagram or architecture explanation.
65. Tech stack.
66. Local setup.
67. Environment variables.
68. Database setup and migrations.
69. RLS policy setup.
70. Seed/demo data instructions.
71. How to run frontend.
72. How to run backend.
73. How to run tests.
74. How to run scheduled jobs locally or simulate them.
75. Billing setup or billing simulation instructions.
76. Live demo URL.
77. Known limitations.
78. Future improvements.

25. Candidate Deliverables

Deliverable	Required content
GitHub repository	Complete source code with clean structure and commit history
README.md	Complete setup, architecture, environment, test, deployment and usage instructions
Live demo	Vercel-hosted working frontend and connected Supabase backend
.env.example	All required configuration keys, without real secrets

Database migrations	Schema and RLS policies reproducible from a clean database
Seed/demo data	Optional but strongly recommended for reviewer usability
Testing	Automated tests for critical flows
Architecture notes	Short explanation of major technical decisions and trade-offs

26. Assessment Rubric

Area	Weight	Evaluation focus
Multi-tenancy & RLS	20%	Correct tenant isolation, policies, RBAC, secure data access
Booking & availability	20%	Correct slot calculation, overlap handling, concurrency safety
Backend/API	15%	Architecture, validation, authorization, error handling, maintainability
Frontend/UX	15%	Booking experience, dashboard, responsive quality, state handling
Database design	10%	Schema quality, integrity, indexes, migrations
Billing & jobs	8%	Subscription state, webhook handling, reminders/idempotency
Testing	7%	Coverage of critical and security-sensitive paths
Documentation & deployment	5%	README, .env.example, Vercel/Supabase setup

26.1 Bonus scoring

- AI scheduling assistant that uses authoritative availability.
- No-show risk scoring with explainable features.
- AI-generated follow-up messages.
- Advanced provider/resource scheduling.
- Strong observability and audit tooling.
- Polished UI and exceptional mobile booking experience.

27. Suggested Implementation Plan

Phase	Work	Suggested output
1. Foundation	Repo, React app, Node API, Supabase project, environment config	Running skeleton
2. Auth & tenancy	Supabase Auth, business creation, membership, RLS	Secure tenant boundary
3. Core data	Services, providers, hours, blocked periods, migrations	Configurable business
4. Availability	Slot generation + overlap logic	Reliable availability API
5. Booking	Public flow + transactional creation	End-to-end booking
6. Dashboard	Calendar + booking actions	Business operations UI
7. CRM	Client profiles/history/metrics	Customer management
8. Notifications	Confirmation + reminder job	Automated communication
9. Billing	Subscription + webhook state	Access control
10. Hardening	Tests, security review, performance, docs	Submission-ready SaaS

27.1 Recommended priority

Do not begin with AI. First make authentication, RLS, availability, booking consistency, and the primary dashboard reliable. Optional features should be added only after the core product is stable.

28. Acceptance Criteria

79. A new business owner can register and create a tenant.
80. The owner can configure a business slug and publish a public booking page.
81. The owner can create at least one service with duration and price.
82. The owner can configure working hours.
83. A public client can select a service and date and see valid slots.
84. The client can complete a booking.
85. The server revalidates availability before insertion.
86. The system prevents double-booking for the same provider/resource.
87. The business dashboard shows the booking.
88. The business can cancel and reschedule the booking.
89. The customer appears in CRM with booking history.
90. A reminder record/job can be generated for eligible appointments.
91. Subscription state can gate premium dashboard access.
92. RLS prevents cross-tenant access.
93. The project can be deployed from documented instructions.
94. A clean reviewer can follow README instructions and reproduce the application.

29. Bonus Challenges

- Multiple providers with different working hours and service capabilities.
- Recurring availability exceptions and holidays.
- Business-level and provider-level time zones.
- Waitlist when no slots are available.
- Cancellation windows and configurable policies.
- Calendar export / iCal integration.
- Customer self-service rescheduling via secure token.
- Analytics dashboard with booking/revenue trends.
- Webhook-driven billing lifecycle with grace periods.
- Background queue abstraction for high-volume notifications.
- AI scheduling assistant with tool-calling and confirmation.

30. Final Submission Checklist

□ GitHub repository is accessible to the reviewer.

□ Production URL is live on Vercel.

□ Supabase project is connected and migrations are documented.

□ Supabase Auth works for business users.

□ At least two test tenants can be created.

□ RLS has been tested for cross-tenant isolation.

□ Public /booking/:business-slug flow works.

□ Service duration and pricing are visible.

□ Availability excludes existing bookings and blocked periods.

□ Concurrent booking protection is implemented or clearly documented.

□ Dashboard calendar works.

□ Booking status actions work.

□ CRM works and remains tenant-scoped.

□ Reminder job works or is clearly simulated.
□ Subscription state and feature gating work.
□ Webhook verification is implemented if using real billing.
□ .env.example is present and contains no real secrets.
□ README contains complete setup instructions.
□ Tests cover critical booking/security behavior.
□ Known limitations and trade-offs are documented.

END OF ASSESSMENT SPECIFICATION